



Data Foundations

AICB-P1 · Ngày 7 · Embedding & Vector Store

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ..

“Agent trả lời sai vì model yếu, hay vì nó không có đúng dữ liệu để suy luận? Ví dụ: Agent CS dùng GPT-4 nhưng trả sai chính sách hoàn tiền — vì data là policy cũ 2023.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

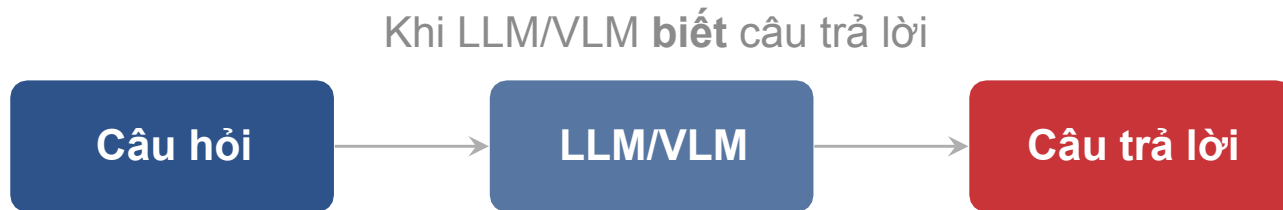
1. Data strategy cho AI product
2. Agent memory architecture
3. Chunking
4. Embeddings
5. Vector store
6. Data processing pipeline
7. Kết nối agent với data
8. Lab 7 + deliverable

- Hiểu tầm quan trọng của dữ liệu cho AI — **data quality** thường quyết định hơn model selection
- Phân biệt được **knowledge data, operational data, contextual data**
- Hiểu **embedding** là lớp biểu diễn nghĩa, không chỉ là một API call
- Hiểu được các bước và phương pháp xử lý dữ liệu trước khi đưa vào AI
- Hiểu **vector store** lưu gì, tìm gì, và lọc bằng metadata ra sao
- Mô tả được pipeline **Document** → **Chunk** → **Embed** → **Store** → **Query** → **Inject**
- Build được một **mini retrieval integration** để nối agent với dữ liệu riêng

Data Strategy Cho Sản Phẩm AI

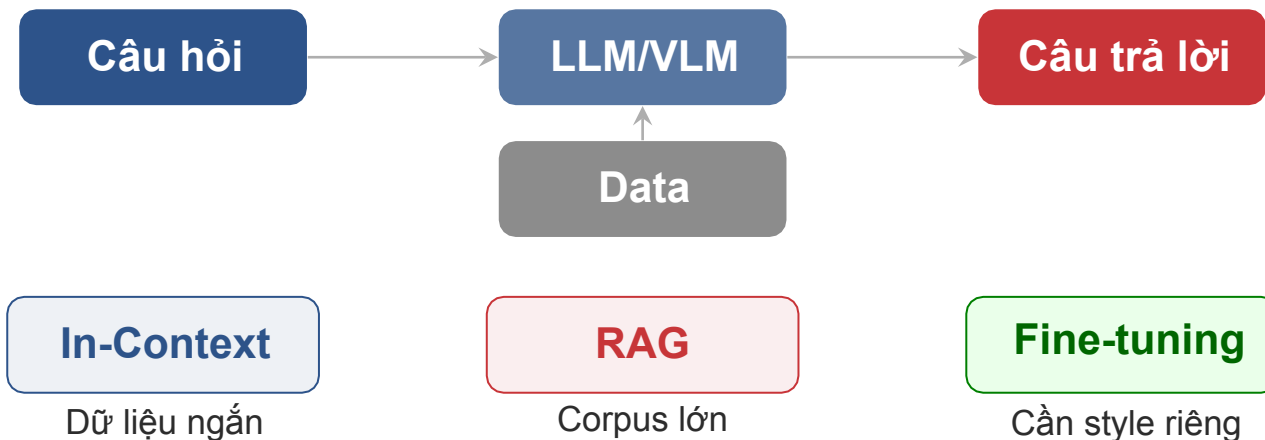
Không phải dữ liệu nào cũng nên đưa vào agent; chọn đúng dữ liệu quan trọng hơn nạp thật nhiều

1



Lưu ý: Đơn giản — nhưng điều này **hiếm khi đủ** cho sản phẩm thật.

Khi LLM/VLM không có sẵn câu trả lời



Day 07 tập trung vào nhánh **RAG**: đưa đúng dữ liệu vào agent thay vì phải đổi model.

Câu hỏi: “Chính sách hoàn tiền của công ty áp dụng bao lâu?” — công ty có 800 file policy

1. In-Context

Dán thẳng nội dung vào prompt:

“Dựa vào tài liệu sau: *[toàn bộ policy]* —
Trả lời: ...”

Không có bước embed, không có DB.

- + Làm được trong 5 phút
- 800 file không nhét nổi vào context window

Dùng khi: 1–20 trang, ít thay đổi.

2. RAG

Chỉ nạp **đoạn liên quan**:

800 file → chunk → embed → store

Query “hoàn tiền” → tìm top-3

→ nhét 3 chunk đó vào prompt

Model **không đổi**, chỉ đổi con-text.

- + Sửa policy → re-index, xong
- + Trích được nguồn để cite
- Cần pipeline + vector store

Đây là Day 07 + Day 08.

3. Fine-tuning

Train lại trọng số model:

5,000 cặp Q–A → training job

→ model mới “thuộc” cách trả lời

Kiến thức nằm **trong** model, không nằm ở prompt.

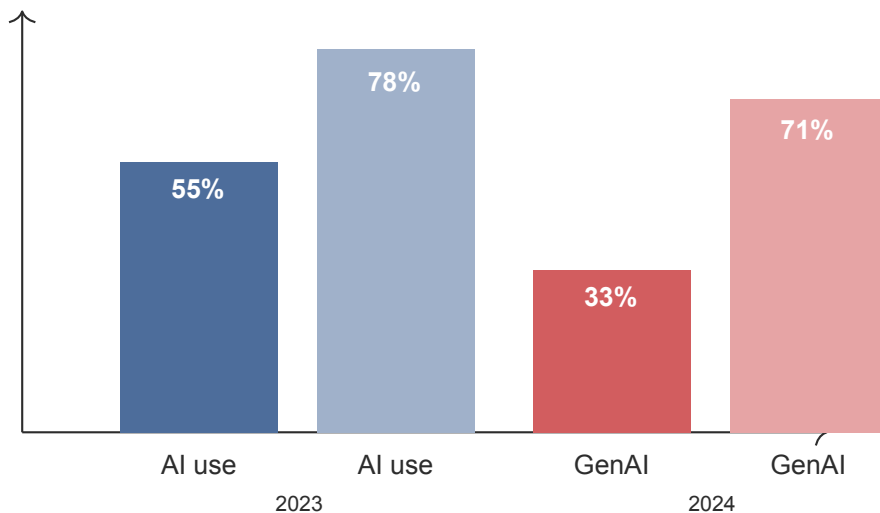
+ Học được **giọng điệu**, format riêng

- Policy đổi → phải train lại
- Tốn GPU, cần data sạch

Dùng khi: cần style/tone, không phải để nhớ fact.

Lưu ý: Hiểu nhầm phổ biến là dùng **finetuning để nạp kiến thức mới**. Finetuning dạy model *cách nói*; RAG cung cấp *điều cần nói*. Fact thay đổi thường xuyên → chọn RAG.

% tổ chức áp dụng



Based on Stanford AI Index 2025.

- AI adoption đã tăng rất nhanh.
- Câu hỏi chuyển từ “dùng model nào” sang “agent được phép biết gì”.
- Nếu dữ liệu sai, cũ, bẩn, hoặc không truy được đúng lúc, output sẽ yếu dù model mạnh.

Lưu ý: **Data quality** và **retrieval quality** thường quyết định trải nghiệm thật hơn là đổi sang model đắt hơn.

Knowledge Data

Tài liệu, policy, SOP, FAQ, manual, hợp đồng, bài viết nội bộ.

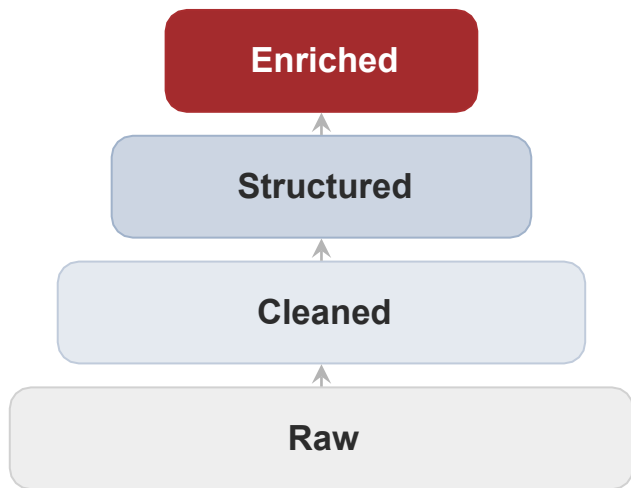
Operational Data

Database, trạng thái đơn hàng, ticket, CRM records, logs, transactions.

Contextual Data

Session history, user profile, preferences, recent actions, channel context.

Knowledge data phù hợp với retrieval; **operational data** thường cần query có kiểm soát; **contextual data** nên được inject ngắn và đúng lúc.



Tầng	Ví dụ cụ thể
Raw	PDF scan lệch, OCR ra “ đổi trả ” thành “ d<i>o</i>i trả ”, HTML chứa tag rác
Cleaned	“ đổi trả ”, bỏ header/footer, chuẩn hóa Unicode
Structured	chunk theo heading, gắn source refund-v3.pdf
Enriched	tag category:support, access:public, quality:verified

Lưu ý: Sai lầm phổ biến: index ngay dữ liệu **raw** rồi kỳ vọng retrieval sẽ tự chữa mọi vấn đề.

- ☒ **Ai sở hữu dữ liệu?** team nào chịu trách nhiệm cập nhật?
- ☒ **Ai được phép truy cập?** dữ liệu nào cần ACL, dữ liệu nào public nội bộ?
- ☒ **Data freshness** bao lâu phải re-index?
- ☒ **PII / sensitive fields** có cần mask trước khi embed không?
- ☐ **Không index mọi thứ theo kiểu “cứ nạp hết vào vector DB đã”.**

Ví dụ: FAQ do team CS sở hữu, public nội bộ, re-index mỗi tuần, không chứa PII.

Agent Memory Architecture

Memory không chỉ là lưu nhiều hơn; nó là quyết định xem agent được nhớ gì, quên gì, và truy gì khi cần

2

Short-term Memory

- Nằm trong context window
- Giữ lịch sử gần đây và task hiện tại
- Rẻ để dùng, nhưng dễ đầy token
- Phù hợp cho session logic ngắn

Long-term Memory

- Nằm ngoài context window
- Thường là vector store, DB, profile store
- Phải truy xuất khi cần
- Phù hợp cho tri thức tích lũy và user history chọn lọc

Context window không phải là vector store. Agent chỉ “nhớ dài hạn” khi có cơ chế truy xuất hoặc lưu trữ bên ngoài.

Chunking

Chunk là đơn vị nhỏ nhất mà agent thật sự truy xuất; cắt sai ở đây thì mọi bước sau đều trả giá

3

1 file PDF (nhiều section):
Chính sách đổi trả...
Chính sách hoàn tiền...
Giao hàng...

↓ chunking

Chunk 1: đổi-trả (150 tok)

Chunk 2: hoàn-tiền (120 tok)

Chunk 3: giao-hàng (90 tok)

Chunking = chia tài liệu dài thành các đoạn nhỏ hơn để embed và index riêng.

Vì sao phải chunk?

- Embedding model có giới hạn token input
- 1 file dài → 1 vector: chỉ tìm được file, không tìm được đoạn liên quan
- Chunk nhỏ → retrieval chính xác hơn, inject ít nhiễu hơn
- **Đưa quá nhiều context không liên quan có thể làm giảm chất lượng câu trả lời** — model bị phân tán, tốn thêm token và latency

Chiến lược phổ biến: theo heading / section (phổ biến nhất); theo số token cố định (200–500); theo câu / paragraph.

Chunk quá to

Dính nhiều chủ đề vào cùng một vector, retrieve trùng nhưng inject rất nhiều.

> 1000 tokens

Chunk hợp lý

Một ý / một section trọn vẹn, có overlap 10–15% giữa các chunk.

200–500 tokens

Chunk quá nhỏ

Mất ngữ cảnh, retrieve nhiều mảnh rời rạc, khó tổng hợp.

< 50 tokens

Khi cắt chunk, lấy thêm 1–2 câu cuối của chunk trước làm phần đầu chunk sau. Giúp giữ ngữ cảnh tại điểm cắt.

Rule of thumb: bắt đầu đơn giản với chunk theo section / heading, rồi tối ưu sau bằng eval thay vì đoán cảm tính.

Chunk quá to

Q1: Hoàn tiền mất bao lâu?

A1: 7 ngày.

Q2: Đổi size được không?

A2:

Trong 30 ngày.

Q3: Ship quốc tế? A3: Không.

Query “đổi size” → retrieve
cả 3 → LLM bị nhiễu bởi Q1,
Q3.

Chunk hợp lý

Q2: Đổi size được không?

A2: Được, trong vòng 30
ngày kể từ ngày mua.

Query “đổi size” → retrieve
đúng Q2 → LLM trả lời chính
xác.

Chunk quá nhỏ

“Trong 30 ngày.”

Query “đổi size” → retrieve
“Q2: Đổi size được không?”
→ thiếu ngữ cảnh, LLM
không có đủ thông tin để trả
lời

Strategy	Cách cắt	Mạnh	Yếu
Fixed-size	Cắt mỗi N token, over-lap 10–15%	Đơn giản, chunk đều, dễ kiểm soát cost	Cắt giữa câu, giữa bảng
Recursive	Thử tách theo $\backslash_n \backslash_n \rightarrow \backslash_n \rightarrow$ câu, đến khi vừa size	Tôn trọng ranh giới tự nhiên; mặc định tốt	Vẫn cắt nếu 1 đoạn quá dài
Heading /structural	Cắt theo heading Markdown ($\#$, $\#\#$) hoặc theo section	Chunk trọn ý, metadata section rõ	Cần tài liệu có cấu trúc tốt

*Ba strategy **nền tảng** — lab yêu cầu implement cả ba rồi so sánh trên cùng một tập query, để thấy chunking là quyết định thiết kế, không phải tham số mặc định.*

Strategy	Cách cắt	Mạnh	Yếu
Semantic	Embed từng câu, cắt tại chỗ cosine giữa 2 câu liền kề tự mạnh (topic shift)	Ranh giới đúng theo ý, không phụ thuộc heading	Tốn embedding lúc ingest; cần tune ngưỡng
Sentence -window	Index 1 câu , nhưng inject câu đó $\pm k$ câu xung quanh	Retrieve rất chính xác, inject vẫn đủ ngữ cảnh	Số record lớn; cần lưu vị trí câu
Proposition / atomic	Dùng LLM viết lại đoạn thành các mệnh đề độc lập , mỗi mệnh đề 1 chunk	Mỗi chunk tự đủ nghĩa, ít nhiễu nhất	Đắt nhất; LLM có thể bóp méo nội dung

Nhóm này **tôn trọng ngữ nghĩa** thay vì đếm token. Đắt hơn khi ingest (cần embed hoặc gọi LLM), nhưng thường tăng retrieval precision rõ rệt trên tài liệu dài, ít cấu trúc.

Strategy	Cách cắt	Mạnh	Yếu
Parent-child	Index chunk nhỏ để search, inject chunk cha lớn hơn	Cân bằng precision và ngữ cảnh; rất phổ biến trong production	Cần lưu quan hệ cha-con
Hierarchical	Nhiều tầng: doc summary → section → paragraph, search từ trên xuống	Tốt cho corpus rất lớn, hỗ trợ câu hỏi tổng hợp	Pipeline phức tạp, phải sinh summary
Code-aware (AST)	Cắt theo hàm / class bằng AST parser	Không bao giờ cắt giữa hàm; giữ import, signature	Cần parser theo từng ngôn ngữ
Table / layout-aware	Giữ nguyên bảng, kèm lại header dòng cho mỗi phần bảng	Bảng không bị mất nghĩa khi cắt	Cần layout parser cho PDF

*Nhóm **cấu trúc & phân cấp**: dùng khi tài liệu không phải văn bản chảy — code, bảng biểu, hợp đồng dài, hoặc khi cần cả tóm tắt lẫn chi tiết.*

- Tài liệu có heading / Markdown tốt → **Heading / structural** (bắt đầu ở đây)
- Không có cấu trúc, văn bản chảy dài → **Recursive**, rồi thử **Semantic**
- Retrieval trùng chunk nhưng LLM *thiếu ngữ cảnh* → **Parent-child** hoặc **Sentence-window**
- Retrieval trùng chunk nhưng LLM *bị nhiễu* → chunk nhỏ hơn, hoặc **Proposition**
- Corpus rất lớn, có câu hỏi tổng hợp → **Hierarchical**
- Source code → **Code-aware (AST)**; PDF nhiều bảng → **Table-aware**

Bắt đầu bằng strategy đơn giản nhất chạy được, dựng bộ eval query, rồi mới nâng cấp **dựa trên số đo** — không nhảy thẳng vào Proposition vì nghe hiện đại.

Trong lab hôm nay chỉ cần 3 strategy nền tảng. Các strategy còn lại là hướng mở rộng cho Day 08 và project cuối phase.

```
import re
# 1. Fixed-size + overlap: đơn giản nhất. Bước = size - overlap => lùi lại giữ nguyên cạnh điểm cắt
def fixed_size(text, size=400, overlap=50):
    w = text.split()
    return [" ".join(w[i:i + size])
            for i in range(0, len(w), size - overlap)]
# 2. Recursive: ưu tiên nhanh gọn tự nhiên
def recursive(text, size=400):
    for sep in ["\n\n", "\n", ". "]:
        parts = [p for p in text.split(sep) if p.strip()]
        if all(len(p.split()) <= size for p in parts):
            return parts
    return fixed_size(text, size) # đoạn quá dài => fallback
# 3. Heading: cắt theo section Markdown, giữ metadata
def by_heading(md):
    s = re.split(r"^(?m){1,3} .+$", md)
    return [{"title": t.strip("# "), "text": b.strip()}
            for t, b in zip(s[1::2], s[2::2]) if b.strip()]
```

Embeddings

Embedding biến ngôn ngữ thành không gian toán học để máy có thể so sánh nghĩa thay vì chỉ so chữ

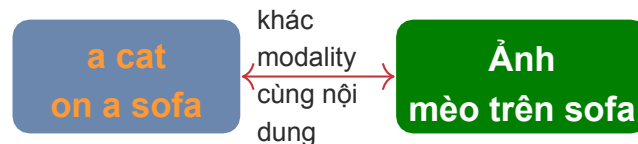
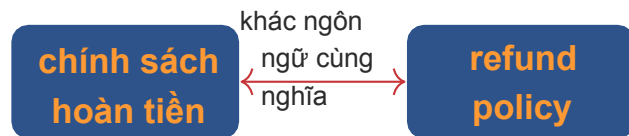
4

HÃY SUY NGHĨ..

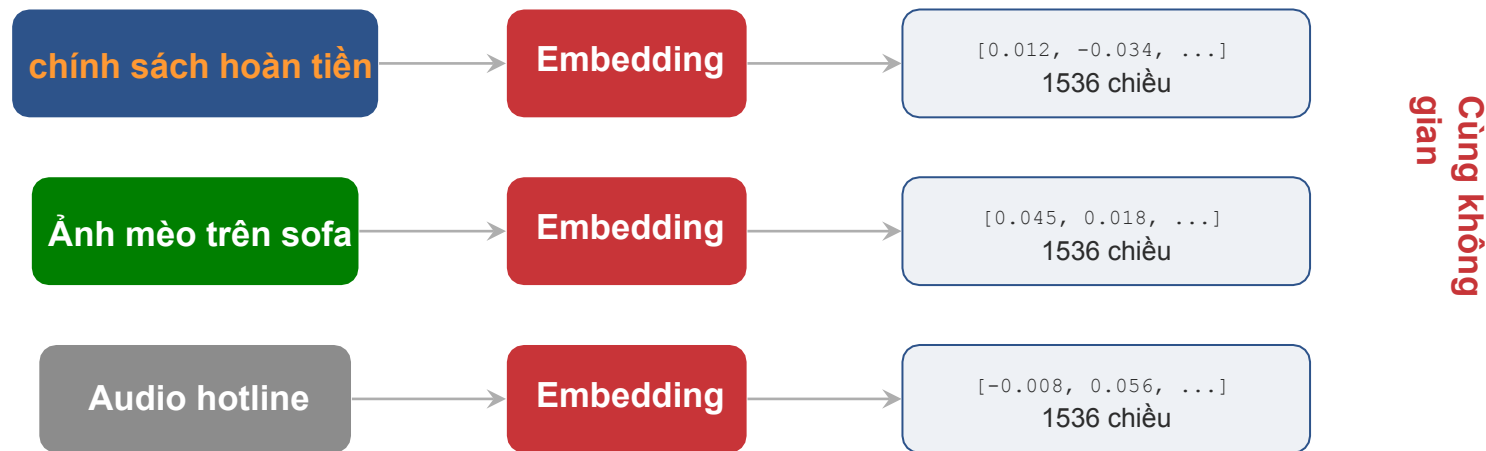
“Làm sao để máy biết hai thứ “giống nhau” — khi chúng không cùng ngôn ngữ, không cùng định dạng?”



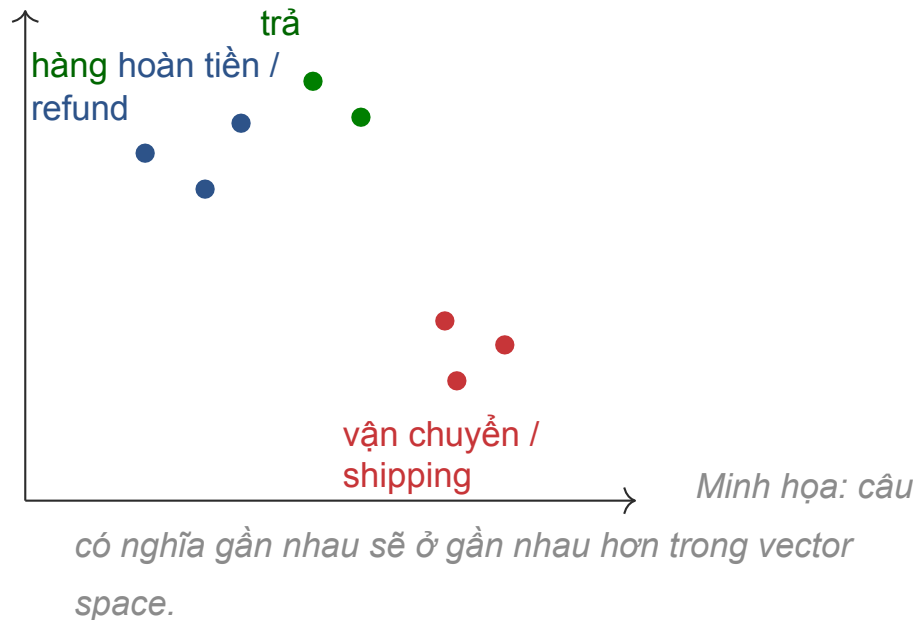
Giữ câu hỏi này trong đầu khi học bài hôm nay



Con người nhìn là biết “giống nhau”. Nhưng máy cần biến text, ảnh, audio thành **số** trong cùng không gian — rồi mới đo khoảng cách được.



Embedding model là hàm biến dữ liệu thô (text, ảnh, audio) thành vector số **cùng kích thước**. Sau đó, đo khoảng cách giữa các vector = đo “độ giống nhau” về nghĩa.



- Text được biến thành vector nhiều chiều.
- Từ đó, máy có thể đo **độ gần về nghĩa**.
- Đây là nền tảng cho semantic search, clustering, dedup, recommendation.

Thập niên 1990s

One-Hot / BOW

Biểu diễn từ thưa (sparse), kích thước bằng từ điển. Không học được ngữ nghĩa và quan hệ giữa các từ.

Năm 2018

ELMo / CoVe

Embedding ngữ cảnh đầu tiên dùng mạng LSTM. Một từ có thể có nhiều vector tùy vào câu chứa nó.

Năm 2013

Word2Vec / GloVe

Kỹ nguyên vector dày (dense). Biến từ thành vector tĩnh. Các từ cùng ngữ cảnh sẽ đứng gần nhau.

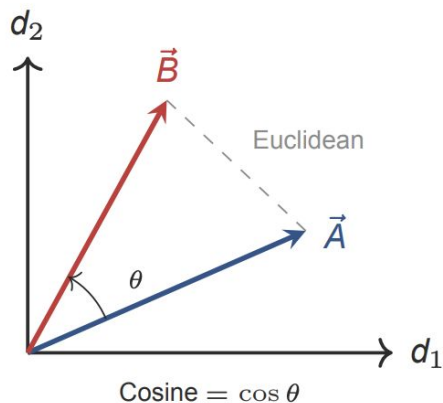
Năm 2019 - Nay

BERT / GPT

Kiến trúc Transformer với Self-Attention vượt trội. Hiểu ngữ cảnh đa chiều, làm nền tảng cho LLM hiện đại.

Sự tiến hóa từ vector thưa, tĩnh không ngữ cảnh sang vector dày đa chiều chứa đầy đủ thông tin ngữ nghĩa là bước ngoặt quyết định của xử lý ngôn ngữ tự nhiên (NLP).

Mô Hình / Thời Kỳ	Ưu Điểm (Pros)	Nhược Điểm (Cons)
Thập niên 1990s One-Hot / BOW	- Cực kỳ đơn giản, dễ triển khai trực tiếp. - Không cần huấn luyện phức tạp, tốc độ xử lý nhanh.	- Ma trận thưa (sparse), tốn tài nguyên bộ nhớ khi từ điển lớn. - Hoàn toàn không hiểu ngữ nghĩa & mối quan hệ giữa các từ.
Năm 2013 Word2Vec/GloVe	- Vector dày (dense) tối ưu không gian lưu trữ vượt trội. - Học được quan hệ ngữ nghĩa (các từ cùng ngữ cảnh đứng gần nhau).	- Vector tĩnh: một từ chỉ có duy nhất một biểu diễn vector. - Không giải quyết được hiện tượng từ đa nghĩa (homonyms).
Năm 2018 ELMo / CoVe	- Embedding động đầu tiên: thay đổi vector theo ngữ cảnh câu. - Giải quyết tốt bài toán từ đa nghĩa nhờ mạng LSTM.	- Mạng LSTM xử lý tuần tự nên tốc độ huấn luyện chậm. - Khả năng ghi nhớ ngữ cảnh dài (long-range dependency) còn hạn chế.
2019 - Nay BERT / GPT	- Cơ chế Self-Attention vượt trội giúp hiểu ngữ cảnh đa chiều. - Làm nền tảng cực mạnh cho các LLM hiện đại ngày nay.	- Yêu cầu tài nguyên tính toán khổng lồ (GPU) để chạy và finetune. - Chi phí triển khai cao và độ trễ (latency) lớn hơn các thế hệ trước.
 Nhận xét: Sự phát triển đi từ đơn giản đến phức tạp, đánh đổi chi phí tài nguyên để lấy khả năng hiểu sâu sắc ngữ nghĩa của ngôn ngữ.		



Cosine similarity — đo hướng

$$\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$$

Mẫu số chia cho hai độ dài → **norm bị triệt tiêu**. 1 = cùng hướng, 0 = vuông góc, -1 = ngược.

Euclidean distance — đo độ lớn

$$d(\vec{A}, \vec{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Khoảng cách “đường chim bay”; 0 = trùng nhau. **Có** tính norm → vector dài/ngắn cho kết quả khác nhau.

1536 chiều: công thức y hệt, chỉ nhiều i hơn — không cần tự code, nhưng cần hiểu score 0.87 vs 0.31 nghĩa là gì.

Câu hỏi gốc: “độ lớn của vector có mang thông tin cần so sánh hay không?”

Dùng COSINE khi...

Chỉ quan tâm **nội dung / hướng**, không quan tâm độ lớn — dữ liệu dài ngắn rất khác nhau nhưng cùng chủ đề. Vector thưa, nhiều chiều: text embedding, TF-IDF, user preference.

Ví dụ: “hoàn tiền” (5 từ) vs cả trang policy hoàn tiền → cosine vẫn thấy giống nhau.

Dùng EUCLIDEAN khi...

Độ lớn là thông tin thật, không phải nhiễu, và các chiều cùng đơn vị / cùng scale. Cần khoảng cách tuyệt đối: tọa độ, cảm biến, image feature, cluster-ing (k-means).

Ví dụ: hai điểm GPS — 2 km và 200 km khác nhau thật, không được chuẩn hóa mất.

Khi vector đã **normalize** về độ dài 1, cosine và Euclidean **xếp hạng giống nhau**. Phần lớn embedding model hiện đại đều trả vector đã normalize → mặc định cứ dùng **cosine** cho text.

Câu A	Câu B	Cosine	Nghĩa
“Chính sách hoàn tiền”	“Quy định đổi trả”	~0.87	Rất gần nghĩa
“Chính sách hoàn tiền”	“Cách giao hàng nhanh”	~0.52	Liên quan
“Chính sách hoàn tiền”	“Thời tiết hôm nay”	~0.31	Không liên quan
“Chính sách hoàn tiền”	“Refund policy”	~0.82	Cross-lingual!

*Embedding model hiểu nghĩa **cross-lingual**: “hoàn tiền” và “refund” gần nhau dù khác ngôn ngữ. Đây là sức mạnh so với keyword search.*

Use case	Cách hoạt động	Ví dụ thực tế
Semantic search	Embed query + embed tài liệu, tìm cosine cao nhất	User hỏi “trả hàng” → tìm được chunk “chính sách đổi trả” dù không trùng từ
Clustering	Gom các vector gần nhau thành nhóm	10,000 ticket CS → tự phân thành 15 chủ đề
Dedup	So cosine giữa các cặp, đánh dấu cặp > threshold	Phát hiện 2 bài FAQ nội dung gần giống, gộp lại
Recommendation	Embed user intent + embed item, rank theo cosine	User vừa đọc “React hooks” → gợi ý “us-eState patterns”

Embedding không tự tạo ra “sự thật”. Nếu text nguồn sai, thiếu, hoặc chunk xấu thì retrieval vẫn tệ — garbage in, garbage out.

```
from openai import OpenAI
import numpy as np

client = OpenAI()
texts = ["Chính sách hoàn tiền", "Quy định đổi trả", "Thời tiết hôm nay"]

resp = client.embeddings.create(
    model="text-embedding-3-small", input=texts
)
vecs = [np.array(d.embedding) for d in resp.data]

# Cosine similarity: nghĩa gần => score cao
cos = lambda a, b: a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(cos(vecs[0], vecs[1]))    # ~0.87 hoàn tiền <-> đổi trả
print(cos(vecs[0], vecs[2]))    # ~0.31 hoàn tiền <-> thời tiết
```

Model	Dim	MTEB Avg	Khi nào dùng
text-embedding-3-small	1536	62.3	Demo, PoC, lab (rẻ, nhanh)
text-embedding-3-large	3072	64.6	Production cần chất lượng
Open-source (e5, BGE)	768–4096	61–66	Self-host, data nhạy cảm, free

Số liệu MTEB tham khảo (đầu 2026); leaderboard thay đổi liên tục — luôn kiểm tra bản mới nhất.

- Không có model “tốt nhất” cho mọi use case. Chọn theo quality, latency, storage, language.
- Với product thực tế, model cân bằng thường tốt hơn model mạnh nhất nhưng chậm và đắt.

Có hàng trăm embedding model (Cohere, Voyage, Gemini, Jina...). Xem bảng xếp hạng đầy đủ: MTEB Leaderboard trên Hugging Face.

Vector Store

Vector store không chỉ lưu vector; nó lưu cả nguồn, metadata, và khả năng lọc để retrieval có ngữ cảnh đúng

5

Mỗi record gồm 4 thành phần

ID

Original chunk (text gốc)

Embedding vector

Metadata (key-value)

Search results (top-k)

Output khi query, không phải thứ được lưu

Ví dụ 1 record thật

Trường	Giá trị
ID	policy-returns-001
Chunk	“Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên tem.”
Vector	[0.012, -0.034, ...] (1536 số)
Metadata	source: refund-v3.pdf, category: support, updated: 2026-03-01, access: public-internal

Vector dùng để tìm (semantic search). **Chunk** dùng để inject vào prompt cho LLM. Hai thứ khác nhau, lưu cùng nhau.

Trường metadata	Ví dụ	Tác dụng
Source / file name	refund-policy-v3.pdf	truy vết và hiển thị nguồn
Category	finance, support, legal	lọc đúng domain trước khi search
Time / freshness	2026-03-01	tránh dùng nội dung cũ
Access level	public-internal, restricted	giới hạn truy cập
Section / chunk id	returns.section.4	debug và cite chính xác hơn

*Retrieval tốt cần **semantic similarity** kết hợp **metadata filtering**.*

Tham số	Ý nghĩa	Ví dụ
Top-k	Lấy bao nhiêu chunk gần nhất	<code>n_results=3</code> : đủ cho câu hỏi đơn. <code>n_results=10</code> : khi cần tổng hợp nhiều nguồn
Score threshold	Bỏ chunk có cosine thấp hơn ngưỡng	<code>0.7</code> : chỉ giữ chunk thật sự liên quan. <code>0.4</code> : chấp nhận liên quan nhẹ
Attribute filter	Lọc theo metadata trước khi search	<code>where={"category": "support"}</code> : chỉ tìm trong tài liệu CS

Filter theo category trước → semantic search → lấy top-3 có score > 0.6. Retrieval tốt = chunk đúng + metadata đủ + filter hợp lý — không phải đổi model.

Tên	Dạng	Điểm mạnh	Phù hợp khi
Chroma	Embedded server (Python)	Cài 1 dòng pip, API cực gọn, chạy in-memory	Học, prototype, PoC nhỏ — dùng trong lab hôm nay
FAISS	Library (Meta)	Nhanh nhất cho ANN search thuần, nhiều loại index	Cần tốc độ, tự quản lý; không phải DB (không meta-data filter, không persistence sẵn)
Qdrant	Server (Rust)	Filter metadata mạnh, hiệu năng tốt, API rõ ràng	Production tự host, cần filter phức tạp
Milvus	Distributed (LF AI)	Scale tỉ vector, GPU index, cloud-native	Corpus rất lớn, có team hạ tầng
Weaviate	Server (Go)	Hybrid search sẵn, schema + module vectorizer	Cần hybrid keyword + vector
pgvector	Extension cho PostgreSQL	Dùng luôn DB đang có, JOIN với data quan hệ, ACID	Đã có Postgres và corpus vừa phải — lựa chọn “nhàm chán” nhưng rất hiệu quả

*Nhóm **open source**: tự host, không phí license, kiểm soát dữ liệu — nhưng phải tự lo scaling, backup, monitoring. Hầu hết đều có bản managed cloud của chính hãng.*

Tên	Dạng	Điểm mạnh	Lưu ý
Pinecone	SaaS thuần	Managed hoàn toàn, serverless, độ tin cậy cao	Vendor lock-in; chi phí tăng theo số vector
Elasticsearch / OpenSearch	Search engine + vector	Hybrid BM25 + vector rất mạnh, hệ sinh thái lớn	Nặng để vận hành nếu chỉ cần vector
Azure AI Search	Managed (Microsoft)	Tích hợp sẵn pipeline ingest, OCR, hybrid	Gắn với hệ sinh thái Azure
Vertex AI Vector Search	Managed (Google)	Scale rất lớn, độ trễ thấp	Cấu hình index phức tạp hơn
MongoDB Atlas / Redis	DB có thêm vector index	Dùng luôn DB sẵn có, giảm số hệ thống	Tính năng vector ít sâu hơn DB chuyên dụng
File Search API (OpenAI, Anthropic...)	Retrieval as a service	Không cần chunk / embed / store — gọi API là xong	Ít kiểm soát chunking, meta-data, và cost path

Nhóm **commercial / managed**: trả tiền để khỏi lo hạ tầng. Đổi lại là chi phí theo scale và phụ thuộc vendor — cần tính **cost path** trước khi cam kết.

Tiêu chí nên hỏi trước

- Bao nhiêu vector? (10^4 vs 10^9 là hai bài toán khác nhau)
- Có cần **metadata filter** phức tạp không?
- Có cần **hybrid search** (keyword + vector)?
- Dữ liệu có ràng buộc **compliance** / on-prem?
- Team có ai vận hành hạ tầng không?

Đường đi thực dụng

- Học / prototype → **Chroma**
- Đã có Postgres → **pgvector**
- Production tự host → **Qdrant** / **Weaviate**
- Không muốn lo hạ tầng → **Pinecone** / managed
- Cần hybrid + đã có ELK → **Elasticsearch**
- Tỉ vector → **Milvus** / Vertex

Lưu ý: Đổi vector store là việc **tương đối dễ** — interface đều là add / query / filter. Đổi **embedding model** thì phải re-index toàn bộ. Vì vậy đừng dành quá nhiều thời gian chọn store ở giai đoạn đầu.

```
import chromadb

client = chromadb.Client()
col = client.create_collection("policies")

# 1. Add: chunk + metadata -> vector store
col.add(
    ids=["p1", "p2"],
    documents=["Khách hàng có 30 ngày để đổi trả.",
               "Hoàn tiền trong 7 ngày làm việc."],
    metadatas=[{"cat": "returns"}, {"cat": "refund"}],
)

# 2. Query: semantic search
results = col.query(query_texts=["đổi size"], n_results=2)

# 3. Inject: dùng kết quả làm context cho LLM
context = "\n".join(results["documents"][0])
```

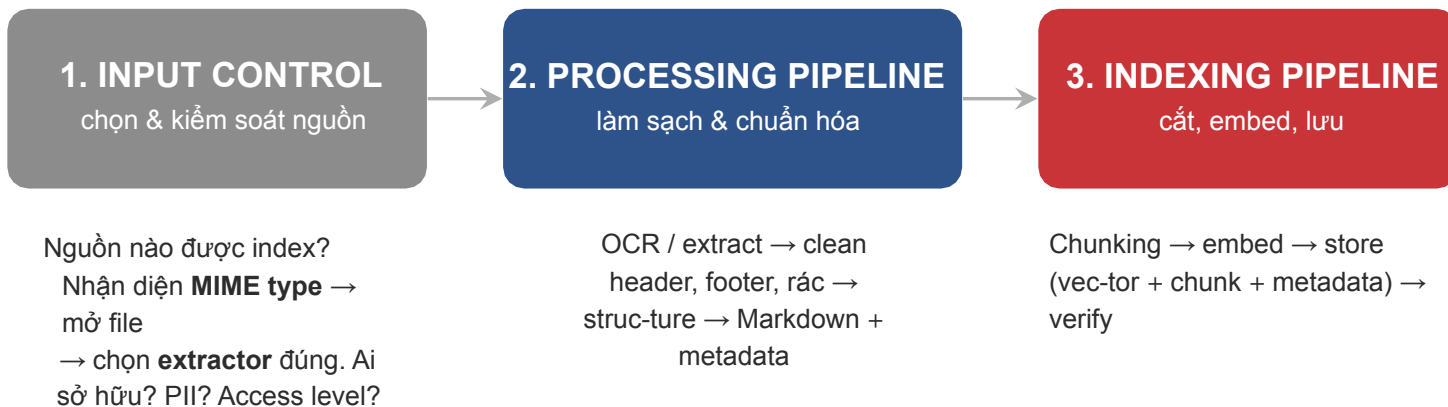
Data Processing

Xử lý dữ liệu để phục vụ cho AI rất quan trọng vì ảnh hưởng trực tiếp đến chất lượng câu trả lời

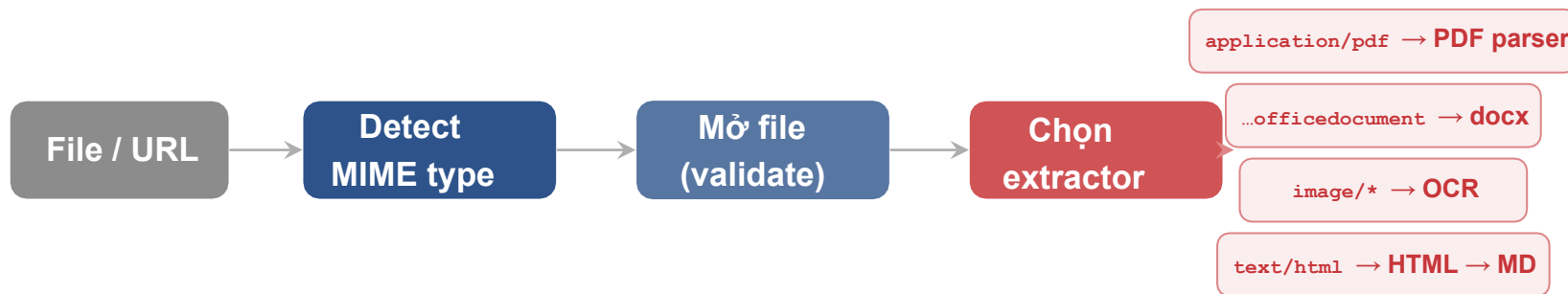
6

Format	Token hiệu quả	Giữ cấu trúc	Khi nào dùng
Markdown	★★★★★	Heading, list, table, bold	Mặc định trong nhiều trường hợp
HTML (clean)	★★★	Bảng lồng, colspan, layout phức tạp	Khi MD mất thông tin cấu trúc
Plain Text	★★★★	Không có cấu trúc	Email, chat log, tran-script
JSON / YAML	★★★	Key-value, nested ob-jects	Structured output, function calling

Nên chọn **Markdown**. So với HTML cùng nội dung, tiết kiệm ~30–50% token vì không có closing tags, attributes, class names. LLM được train trên lượng lớn MD nên hiểu rất tự nhiên.



Lưu ý: Lỗi retrieval thường **không** sinh ra ở khối 3. Nó sinh ra ở khối 1 (index sai nguồn) hoặc khối 2 (bỏ bước clean) — nhưng chỉ **lộ ra** khi query. Debug thì đi ngược từ 3 về 1.

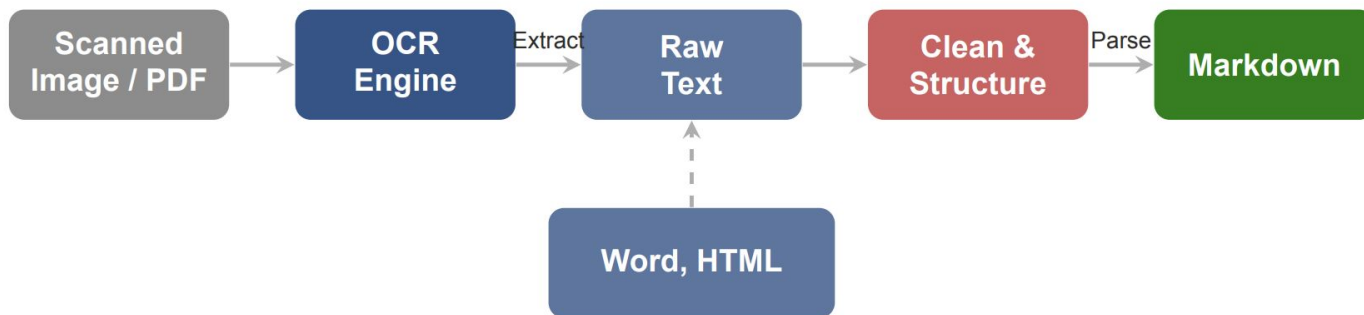


Đừng tin phần mở rộng file

.pdf có thể là scan ảnh; .txt có thể là JSON; .doc có thể là RTF. Kiểm tra **magic bytes** / MIME thật, không chỉ đọc đuôi file.

Fail nhanh, ghi log rõ

Không có extractor → **bỏ qua và log**, đừng index rác. Ghi lại mime, extractor, status vào metadata để debug về sau.



Đường thay thế cho tài liệu có text sẵn

Lưu ý: Sai lầm phổ biến: bỏ qua bước **Clean & Structure**, đưa raw text thẳng vào chunking. Kết quả: chunk chứa header/footer rác, OCR lỗi, ký tự đặc biệt — retrieval quality giảm rõ rệt.

Kết Nối Agent Với Data

Retrieval pipeline là chiếc cầu nối giữa dữ liệu riêng và hành vi của agent

7

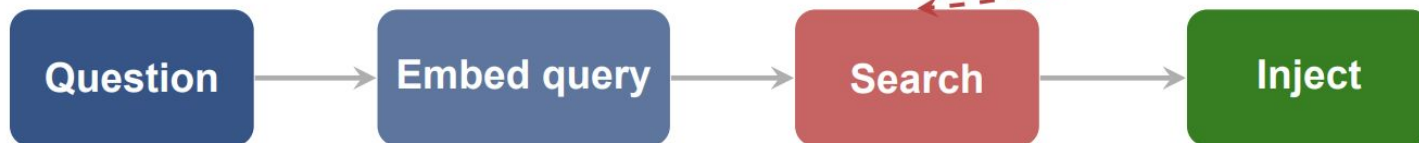
Ingestion

chạy 1 lần



Retrieval

mỗi câu hỏi



Lưu ý: Ingestion chạy **offline** khi dữ liệu thay đổi. Retrieval chạy **online** mỗi khi user hỏi. Hai pha này tách biệt nhau.

Retrieval

- Tìm context liên quan cho câu hỏi hiện tại
- Đọc từ knowledge base
- Trọng tâm: relevance và grounding

*Ví dụ: hỏi “chính sách đổi trả” → tìm trong FAQ
→ trả lời dựa trên tài liệu.*

Memory

- Lưu trạng thái, sở thích, lịch sử chọn lọc
- Từ user profile hoặc interaction history
- Trọng tâm: continuity và personalization

*Ví dụ: nhớ user lần trước đã đổi size M → gợi ý
“đổi lại size khác?”*

Lưu ý: Nhiều hệ thống dùng cả hai: **retrieval** để biết domain facts, **memory** để biết user này là ai và đã làm gì trước đó.

Query: “Chính sách đổi trả áp dụng trong bao lâu?”

Chunk xấu (raw, không section)

Retrieved (cosine 0.61): “...giao hàng miễn phí đơn trên 500k. Đổi trả trong 30 ngày. Liên hệ hotline 1900...”

LLM: “Bạn có thể đổi trả và liên hệ hotline 1900...” (nhiều, thiếu chi tiết)

Chunk tốt (theo section + metadata)

Retrieved (cosine 0.89): “Chính sách đổi trả: Khách hàng có 30 ngày kể từ ngày nhận hàng. Sản phẩm phải còn nguyên tem.”

LLM: “30 ngày kể từ ngày nhận, sản phẩm còn nguyên tem.” (chính xác, có nguồn)

Lưu ý: Cùng model, cùng query — khác nhau hoàn toàn vì **chất lượng chunk**.

Key Takeaways

Day 07 đặt nền cho RAG và mọi hệ thống AI dùng tri thức riêng

8

- 1 **Data quality** thường quan trọng hơn đối sang model đắt hơn.
- 2 **Embedding** là lớp dịch ngôn ngữ sang không gian có thể so sánh nghĩa.
- 3 **Vector store** là bộ nhớ dài hạn có thể tìm kiếm bằng ngữ nghĩa và metadata.
- 4 **Retrieval pipeline** là cầu nối từ dữ liệu riêng tới câu trả lời grounded của agent.

1. Stanford HAI. *The 2025 AI Index Report*.
<https://hai.stanford.edu/ai-index/2025-ai-index-report>
2. OpenAI Docs. *Embeddings Guide*. <https://platform.openai.com/docs/guides/embeddings>
3. OpenAI Docs. *File Search*. <https://platform.openai.com/docs/guides/tools-file-search>
4. Hugging Face. *MTEB Benchmark*. <https://huggingface.co/blog/mteb>
5. Chroma Documentation. <https://docs.trychroma.com/>
6. Karpukhin et al. *Dense Passage Retrieval for Open-Domain QA*. arXiv:2004.04906

Danh sách đầy đủ có trong lab handout.

RAG pipeline

“Ngày 8 đi tiếp sang RAG pipeline: in-dexing, retrieval, generation, evalua-tion.”

- Hoàn thiện `solution.py` nếu chưa pass hết tests
- Rà lại knowledge base: bỏ nội dung nhiều, thêm metadata
- Thử thêm 3 queries khó hơn, tìm failure cases mới